AI - ASSISTED CODING
Assignment - 13.4
2303A510B7
Batch - 14

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

TASK 1: Refactoring Data Transformation Logic

Prompt:

Suggest a more Pythonic way to double all elements in a list instead of using an explicit loop.

Legacy Code:

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Code:

```
numbers = [1, 2, 3, 4, 5]
doubled = [x * 2 for x in numbers]
print(doubled)
```

Output:

```
[2, 4, 6, 8, 10]
```

Explanation:

- The legacy code uses a **for loop** to iterate through the list and append doubled values into another list.
- This approach works but requires more lines of code.
- The refactored code uses a **list comprehension**, which performs the same operation in a single line.
- List comprehensions are more **Pythonic, concise, and readable**.
- Thus, the functionality remains the same while improving code clarity.

TASK 2: Improving Text Processing Code Readability

Prompt:

How can repeated string concatenation be improved in Python?

Legacy Code:

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Code:

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
Refactoring with AI improves quality
```

Explanation:

- The legacy code builds a sentence using **repeated string concatenation inside a loop**.
- Since strings are immutable in Python, each concatenation creates a new string, which is inefficient.
- The refactored code uses the **join() method** to combine all words in the list with spaces.
- This approach is **faster, cleaner, and easier to understand**.
- It also reduces the number of lines in the code.

TASK 3: Safer Access to Configuration Data

Prompt:

Suggest a safer method to access dictionary keys that may not exist.

Legacy Code:

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Code:

```python
config = {"host": "localhost", "port": 8080}
timeout = config.get("timeout")
if timeout is not None:
    print(timeout)
else:
    print("Default timeout used")
```

Output:

```
Default timeout used
```

Explanation:

- The legacy code checks whether a key exists in a dictionary using an **if condition** before accessing it.
- This makes the code slightly longer and less elegant.
- The refactored code uses the **get() method**, which safely retrieves the value of a key.
- If the key does not exist, it returns **None or a default value** instead of causing an error.
- This makes the code **simpler and safer**.

TASK 4: Refactoring Conditional Logic for Scalability

Prompt:

How can multiple if-elif conditions be replaced with a scalable approach?

Legacy Code:

```
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Code:

```
action = "divide"
x, y = 10, 2
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b,
}
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

Output:

```
5.0
```

Explanation:

- The legacy code uses multiple **if–elif statements** to select different operations.
- As more operations are added, the code becomes difficult to maintain.
- The refactored version uses a **dictionary mapping technique**, where operations are stored as key–function pairs.
- This makes the program **more scalable and easier to extend**.
- It also improves code readability.

TASK 5: Simplifying Search Logic in Collections

Prompt:

How can searching for an element in a list be simplified?

Legacy Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = "eraser" in inventory
print("Item Available" if found else "Item Not Available")
```

Output:

```
Item Available
```

Explanation:

- The legacy code searches for an item in a list using a **loop and a flag variable**.
- This increases the number of lines and makes the logic slightly complex.
- The refactored code uses Python's **membership operator (in)** to directly check if the item exists in the list.
- This reduces the code length and improves readability.
- The program still produces the same output.